Cryptography Fundamentals and Pentesting Basics

[Python code solution with GitHub]

---

1. Caesar Cipher:

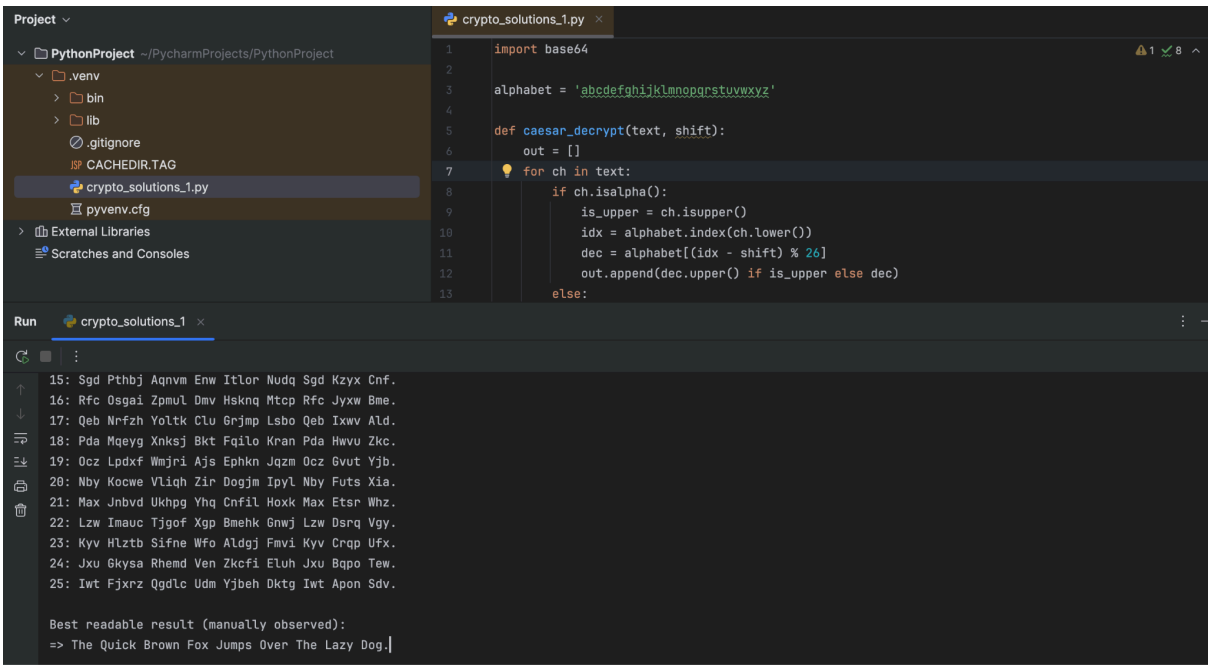
Task: decrypt given cyphertext: “Hvs Eiwqy Pfckb Tcl Xiadg Cjsf Hvs Zonm Rcu.”

- Challenge: Perform Frequency Analysis or brute-force attack to decrypt a ciphertext. Provide python code solution with GitHub.
- Discussion: Why is Caesar cipher insecure? Where might legacy systems still use similar encryption?

Decrypted plaintext:
The Quick Brown Fox Jumps Over The Lazy Dog.

Why it's insecure - Only 26 possible shifts, Simple inspection reveals plaintext quickly and offers no resistance against known-plaintext.

Where similar/simple schemes still appear - Educational examples and puzzles for teaching, very old devices or software might use XOR with a fixed key or Caesar-like substitutions as a legacy "protection".



2. XOR Encryption/Decryption:

Step 1: Caesar Cipher Challenge

- Ciphertext: mznxpz
- Challenge: Perform a brute-force or frequency analysis attack to decrypt the Caesar-encrypted text.
- Clue: The ciphertext is an encrypted anagram of the passphrase.

Ciphertext: mznxpz
Brute-force result (shift = 21): rescue

Step 2: Solve the Anagram

- Decrypted Text: Rearrange the decrypted words to form the original passphrase.
- Hint: The final passphrase is a fundamental concept in cryptography.

Anagram of rescue → passphrase: secure

Step 3: XOR Decryption

- Given Ciphertext in base64: Jw0KBlIMAEUXHRdFKyoxVRENEgkPEBwCFkQ=
- Instructions: Use the recovered passphrase from Step 2 to XOR-decrypt the message (first you must convert base64 then decrypt).

Base64 ciphertext: Jw0KBlIMAEUXHRdFKyoxVRENEgkPEBwCFkQ=
Base64 → bytes, XOR key = secure → Decrypted plaintext: This is the XOR challenge!